

Lab0:Gitlab 实验报告

梁心一 25800190015

一、回答问题

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

有，完成一个人工智能课程的大作业，使用 Github 和 git 终端分工协作。

主要开发者创建了一个仓库，另外三个同伴分别建立 fork 来把初始项目加载到本地，并按自己的分工需求进行修改。

2. Git 为什么要设计“暂存-提交”两个步骤？

暂存操作可以进行一个临时备份，可以在小修改出错随时回退某个文件，不需要描述，也不会占用太多空间。

提交操作则是用在进行了一系列修改之后，对项目进行一次永久的保存与备份，方便回溯比较久远的信息，和保存记录比较大的修改。

设计两个步骤能够兼顾开发中频繁的小修改的保存和回退的需求，和长期的大修改的备份，注释和读取的需求。

3. git branch 和 git branch -a。这两条命令的区别是什么？

git branch 只列出当前电脑/WSL 环境本地已存在的分支。可用于快速查看自己当前在哪个本地分支上工作。

git branch -a 列出本地所有分支，以及从远程仓库（如 GitHub）同步过来的远程追踪分支。可用于查看远程仓库有哪些分支，或准备拉取/切换到远程的新分支时。

二、使用模板建立个人仓库

实验步骤（截图略）：

1. 用 template 功能复制模板仓库为个人仓库

2. 利用 SSH 进行本地和云端的连接

3. 利用 git clone 将云端仓库拉取到本地

修改过代码的项目仓库见：

<https://github.com/Lxyfdu/Lab0-GitLab>

修改如下：

```
GitLab > main.c

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("To C or not to C, that is the question.\n");
6      printf("Hello, world!\n");
7  }
```

运行效果如下：

```
● jimmy@Lxy-MeCr:~/ICS2026/test-git/Lab0-GitLab$ make
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
● jimmy@Lxy-MeCr:~/ICS2026/test-git/Lab0-GitLab$ ./main
To C or not to C, that is the question.
Hello, world!
```

三、阅读规范以及思考

1. Commit message 和 Change log 编写指南

Git 每次提交代码，都要写 Commit message，其中一种规范是 Angular 规范。使用 Commit message 能够为 commit 提供更多的历史信息，方便快速浏览或者查找特定信息，也可以利用 commit message 来生成 Change log。Commit message 格式包括 Header, Body 和 Footer 等，大致如下：

```
<type>(<scope>): <subject>
// 空一行
<body>
// 空一行
<footer>
```

文章强调良好的 Commit message 能清晰呈现代码演进脉络、提高 Code Review 效率，并能通过本自动生成规范的 Change log。此外，利用 Commitizen、validate-commit-msg 等辅助工具，也可以建立标准化的提交注释。

2. Gitflow 使用规范

文章详细介绍了基于 Git 的分支管理模型 Gitflow。该模型基于分支管理策略，来避免多人协作过程中代码冲突、代码版本出现问题。

该模型该围绕两条长期主干分支（master 用于发布稳定版本，develop 用于日常开发）以及三类短期分支（feature 开发新功能、release 预发布准备、hotfix 紧急修复）。并给出了相应的使用示例。

3. 对为什么要学习 Git 的理解

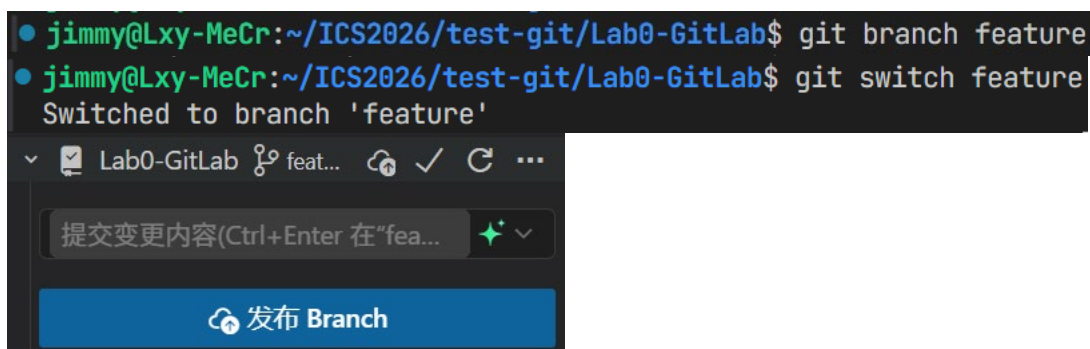
学习 Git 主要是培养现代软件工程中的“版本控制思维”与“团队协作习惯”。为未来的团队开发做准备。

Git 能实现历史备份，注释更新，项目回滚、追踪问题根源等功能，提高复杂且容易出错的大项目的管理容错。结合 Gitflow、规范 Commit 等工具链，Git 将混乱的协作转化为标准化流程，分支隔离了开发风险，规范的日志赋予了代码可读性，清晰的版本号则保障了生态兼容。用一套规范的方式提高不同开发者间的协调沟通效率

因此，Git 是学习计算机的人必备的基础技能，也能够利用 Git 的学习来培养规范的开发习惯。

四、Git 分支管理

1. 创建 feature 分支



2. 分别在 feature 和 main 分支上修改 main.cpp 的句子

feature 分支的修改:

```
Lab0-GitLab > @ main.c

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Time Limit Exceeded.\n");
6      printf("Hello, world!\n");
7  }
```

main 分支的修改:

```
Lab0-GitLab > @ main.c

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Memory Limit Exceeded.\n");
6      printf("Hello, world!\n");
7  }
```

遇到冲突:

```
◎ jimmy@Lxy-MeCr:~/ICS2026/test-git/Lab0-GitLab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

解决冲突:

保留当前 (main) 的修改, 完成合并

```
Lab0-GitLab > @ main.c

传入 98edaa7 · refs/heads/feature      当前 148ab47 · refs/heads/main
3  int main()                          3  int main()
4  {                                    4  {
5      printf("Time Limit Exceeded.\n"); 5      printf("Memory Limit Exceeded.\n");
6      printf("Hello, world!\n");        6      printf("Hello, world!\n");

结果 Lab0-GitLab/main.c                剩余 0 个冲突
3  int main()
4  {
5      printf("Memory Limit Exceeded.\n");
6      printf("Hello, world!\n");
7  }
```

完成合并

结果如下: nothing to commit, working tree clean 表示问题已经解决

```
• jimmy@Lxy-MeCr:~/ICS2026/test-git/Lab0-GitLab$ git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

3. 简要分析

merge 出现错误主要是源于在两个分支都修改了 main.c 的同一行, 此时两边都做了修改并且修改不同, git 无法判断保留哪一处。如果修改不同位置/不会互相影响的位置就不会出错。